# Class 22: Requirements Engineering for ML Systems

## Session 22 of 36 – From Stakeholder Need to Verifiable System Behaviour

Data-driven Systems Engineering

# Roadmap of the class

1. **Requirements and accountability**
   Stakeholder need, system requirement, acceptance evidence, operational check.
2. **Writing verifiable requirements**
   Anatomy of a testable statement; functional, quality, and domain requirements.
3. **Specification and traceability**
   The IEEE software requirements specification (SRS) and traceability.
4. **Workshop**
   A one-page requirements brief for the fraud or the pasteurization case.
5. **Wrap-up**
   References and take-away message.

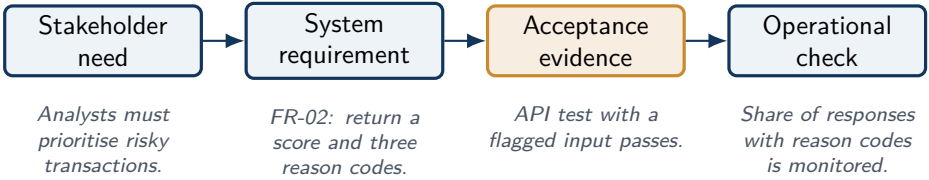
# Requirements and accountability

*What exactly does a team commit to when it accepts a requirement?*

Stakeholder needs, system requirements, acceptance criteria

1. Accountability | 2. Writing | 3. Specification | 4. Workshop | 5. Wrap-up

# Requirements create a chain of accountability




## Why this matters for ML systems

A satisfactory offline metric alone does not establish that the data pipeline, interface, decision workflow, and running system are acceptable.

# Three levels, three audiences

| Level | Purpose | Example |
|---|---|---|
| **Stakeholder need** | Outcome desired by a user or sponsor | Analysts need to prioritise risky transactions. |
| **System requirement** | Behaviour or constraint imposed on the system | The service shall return a risk score and reason codes for each valid request. |
| **Acceptance criterion** | Evidence that demonstrates satisfaction | In the acceptance sample, 95% of valid requests return within 800 ms with three reason codes. |

**Discipline**

Keep the need independent of a particular model; keep the requirement independent of a particular implementation where possible.

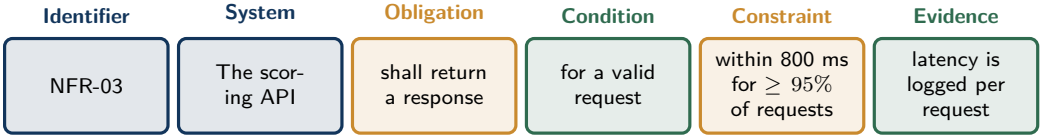
# Writing verifiable requirements

*How do we write a statement that an independent reviewer can pass or fail?*

Anatomy of a requirement, types of obligation, ML uncertainty

1. Accountability | 2. Writing | 3. Specification | 4. Workshop | 5. Wrap-up

# Anatomy of a testable requirement




## Reusable pattern

**[Identifier]** The **[system]** shall **[observable action]** for **[condition/input]** within **[measurable constraint]**, recorded as **[evidence]**.

- Use *shall* for an agreed obligation; avoid "fast", "accurate", "user-friendly", and "as appropriate" unless defined.
- State the condition and the measurement method as well as the target value.

# Requirements address different kinds of obligation

**Functional**

*What must the system do?*

Display the current process state and temperature for a selected batch.

**Quality (non-functional)**

*How well must it do it?*

Raise a critical alert within 3 seconds of a confirmed abnormal state.

**Domain**

*Which external rule or fact constrains it?*

Evaluate temperature only against the calibrated sensor range.

**Key point**

Quality and domain requirements are requirements in their own right: reliability, safety, privacy, and trust are frequently decided there. Examples refer to the pasteurization monitoring case.

# ML systems add conditional and operational uncertainty

## What may vary

- Input distributions and missing data
- Label delay and label quality
- Decision cost across groups or cases
- Latency, load, and service availability

## What requirements must clarify

- Population and operating conditions
- Metric, threshold, and measurement window
- Fallback behaviour and human ownership
- Logging, review, and escalation rules

## Formal caution

"The model shall be accurate" is incomplete until the metric, data slice, threshold, evaluation period, and decision consequence are named.

# From aspiration to acceptance condition

| Vague statement | | Verifiable requirement |
|---|---|---|
| *The dashboard should be reliable.* | $\Rightarrow$ | The dashboard shall reconnect automatically after a dropped stream and show a stale-data indicator within 5 seconds. |
| *The fraud model should be accurate.* | $\Rightarrow$ | On the locked acceptance dataset, the model shall achieve a recall of at least 0.82 at a false-positive rate no greater than 0.12. |
| *Users need explanations.* | $\Rightarrow$ | For every flagged transaction, the API shall return the three highest-ranked contributing features. |

**Review question**

Could an independent reviewer decide whether this statement passed or failed? If not, it is not yet a complete requirement.

# Specification and traceability

*Where do the requirements live, and how do they reach tests and monitoring?*

The IEEE software requirements specification, qualities of a good SRS, traceability

1. Accountability | 2. Writing | 3. Specification | 4. Workshop | 5. Wrap-up

# The IEEE software requirements specification (SRS)

**Software Requirements Specification**
Fraud Scoring Service – version 1.0

**Table of contents**


**1 Introduction**
1.1 Purpose 1.2 Scope
1.3 Definitions, acronyms, and abbreviations
1.4 References 1.5 Overview
**2 Overall description**
2.1 Product perspective 2.2 Product functions
2.3 User characteristics 2.4 Constraints
2.5 Assumptions and dependencies
**3 Specific requirements**
3.1 External interfaces 3.2 Functions
3.3 Performance requirements
3.4 Logical database requirements
3.5 Design constraints
3.6 Software system attributes
**Appendixes** **Index**


- **IEEE Std 830-1998** defines the content and qualities of an SRS; it is superseded by **ISO/IEC/IEEE 29148:2018**, which retains the same content.
- **Sections 1–2** give the reader context: purpose, scope, users, constraints, and assumptions.
- **Section 3** holds the verifiable statements: the part that is reviewed, tested, and traced.
- The outline is a checklist, not a form to fill in mechanically: an absent section should be a decision, not an omission.

# Section 3 of the SRS: specific requirements

| IEEE 830 subsection | What it specifies | Fraud scoring service |
|---|---|---|
| **3.1 External interfaces** | Inputs, outputs, formats, and other systems | JSON request with validated transaction features; JSON response |
| **3.2 Functions** | Actions in response to inputs, including invalid ones | FR-02: score and three ranked reason codes for a high-risk transaction |
| **3.3 Performance** | Static and dynamic numerical requirements | NFR-03: 95% of valid requests answered within 800 ms |
| **3.4 Logical database** | Stored information, retention, integrity | Every prediction logged with model version and timestamp |
| **3.5 Design constraints** | Standards, regulation, hardware limits | Personal data processed under the General Data Protection Regulation (GDPR) |
| **3.6 System attributes** | Reliability, availability, security, maintainability, portability | Fallback to manual review when the model is unavailable |

## ML-specific content

Data requirements (source, version, population), performance per data slice, and monitoring obligations belong in 3.3, 3.4, and 3.6.

# Demonstration: one page of an SRS



## 3.2 Functions

| ID | Requirement | Priority | Verification | Source |
|---|---|---|---|---|
| FR-01 | The service shall reject a request with a missing or out-of-range feature and return an error code that names the feature. | Essential | Test | Analyst lead |
| FR-02 | When a transaction is classified as high risk, the service shall return the score and three ranked reason codes in the JSON response. | Essential | Test | Analyst lead |

## 3.3 Performance requirements

| | | | | |
|---|---|---|---|---|
| NFR-03 | The service shall respond to a valid request within 800 ms for at least 95% of requests in the acceptance workload; latency shall be logged. | Essential | Load test | Operations |
| NFR-04 | On the locked acceptance dataset, recall shall be at least 0.82 at a false-positive rate no greater than 0.12. | Essential | Analysis | Risk office |

**ID**
unique and stable; cited by tests and monitors

**Priority**
ranked for importance (essential, conditional, optional)

**Verification**
test, analysis, inspection, or demonstration

**Source**
the stakeholder or rule the requirement traces to

Section 3 is a set of individually identified statements, not continuous prose: each one can be reviewed, tested, and traced on its own.

# Eight qualities of a good SRS (IEEE 830)

**Correct**
Every requirement is one the software shall meet.

**Unambiguous**
Every requirement has exactly one interpretation.

**Complete**
All significant requirements and responses to all inputs are stated.

**Consistent**
No subset of requirements is in conflict.

**Ranked**
Each requirement carries its importance and/or stability.

**Verifiable**
A finite, cost-effective process can check each requirement.

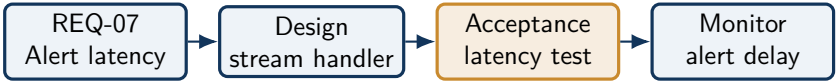
**Modifiable**
Structure and style allow changes without breaking the document.

**Traceable**
The origin of each requirement is clear, and it can be referenced later.

## Link to the workshop

The five peer-review checks used later (necessary, unambiguous, verifiable, feasible, traceable) are a practical subset of these qualities.

# Traceability turns requirements into engineering work




| **Requirement** | Source | Design element | Acceptance test | Operational check |
|---|---|---|---|---|
| **REQ-07** | Plant operator | Stream handler | Injected abnormal state, alert $\leq 3$ s | Alert-delay percentile |
| **DR-01** | Calibration record | Input validator | Boundary-value test | Rate of out-of-range values |
| **FR-02** | Analyst lead | Explanation module | API test with flagged input | Responses with reason codes |

- A traceability matrix makes impact analysis possible when a threshold, data source, interface, or regulation changes.
- Deployment gates and monitoring rules should trace back to a requirement, not be invented after release.

# Workshop

*Can you write a requirements brief that another team can review?*

Discover, specify, decide, define evidence, review

1. Accountability | 2. Writing | 3. Specification | **4. Workshop** | 5. Wrap-up

# The workshop deliverable: a one-page requirements brief

## Required content

- Decision, stakeholder, and system boundary
- Three functional requirements
- Two measurable quality requirements
- One domain constraint or assumption

## Evidence of judgement

- One acceptance method per requirement
- One important tension and a decision
- A short review record from another team

## Choice of case

Use either the fraud scoring service or the pasteurization monitoring dashboard; do not attempt to specify both. The brief is a condensed Section 3 of an SRS.

# Workshop plan and milestones

| | | |
|---|---|---|
| **10 min** | Discover | Identify actors, decision, operating context, and constraints. |
| **12 min** | Specify | Draft three functional and two quality requirements. |
| **6 min** | Decide | Record one material trade-off and its rationale. |
| **10 min** | Review | Exchange briefs and test each statement against the review checklist. |

**Output**

One concise, reviewable requirements brief—not a solution design and not a list of desirable features.

# Step 1: Discover the decision context

**Fraud scoring**

- Who reviews a flagged transaction?
- What decision follows the score?
- What cost follows a false positive or missed case?
- When are labels available for evaluation?

**Pasteurization monitoring**

- Who receives and acts on an alert?
- What must remain visible during a stream failure?
- Which sensor values are valid?
- What response time is operationally useful?

**Instruction**

Write facts, questions, and assumptions separately. An unverified assumption is not yet a requirement.

# Step 2: Establish the system boundary

**Boundary statement template**

The system receives _____, produces _____, and supports _____. It does not decide _____.

**Example**

The fraud scoring service receives validated transaction features, produces a score and reason codes, and supports analyst prioritisation. It does not autonomously block a customer transaction.

- A boundary prevents requirements from extending into policy, model training, or downstream action outside the team's scope.

# Step 3: Draft functional requirements

**Drafting test**

Each statement should identify:

- a condition or input;
- an observable system action;
- an output or stored result.

**Example**

**FR-02** When a transaction is classified as high risk, the service shall return the score and three ranked reason codes in the JSON response.

**Avoid**

"The system shall use SHAP" is a design choice. State the explanation behaviour required; leave the method open unless the method is itself constrained.

# Step 4: Set measurable quality constraints

| **Quality concern** | Specify the measurement, population, target, and evidence |
|---|---|
| **Latency** | Valid requests; percentile and time limit; load profile; service log. |
| **Reliability** | Failure scenario; recovery behaviour and limit; incident or automated test. |
| **Model performance** | Dataset/version and data slice; metric and threshold; evaluation report. |
| **Auditability** | Event fields and retention period; query or log review. |

**Instruction**

Draft two quality requirements. If the target is unknown, record who must set it and what evidence they need—do not choose a number merely to make the sentence look precise.

# Step 5: Record a real trade-off

## Typical tensions

- Recall vs. false-positive workload
- Explanation detail vs. response time
- Audit retention vs. privacy minimisation
- Availability vs. strict input validation

## Decision record

**T-01:** Detailed explanations may increase latency. For initial release, return three precomputed reason codes and preserve the 800 ms latency target because analysts need prompt triage.

## Instruction

Name the competing concerns, the chosen priority, the reason, and the condition that would cause the decision to be revisited.

# Step 6: Define acceptance evidence

| Requirement | Acceptance method | Evidence retained |
|---|---|---|
| **FR-02 reason codes** | API test with flagged input | Response fixture and test result |
| **NFR-03 latency** | Load test against acceptance workload | Latency percentile report |
| **DR-01 sensor range** | Boundary-value test and calibration review | Test output and calibration record |

**Rule**

Every requirement needs a plausible form of evidence. If no evidence can be imagined, revise the statement or challenge whether it belongs in the brief.

# Peer review: five checks

1. **Necessary:** Does it protect a stated stakeholder need or domain constraint?
2. **Unambiguous:** Would two reviewers understand it in the same way?
3. **Verifiable:** Is the condition, target, and evidence clear?
4. **Feasible:** Is it credible within the known system and project constraints?
5. **Traceable:** Can it be linked to an acceptance test or operational check?

## Review protocol

Exchange briefs. Mark one statement that passes cleanly and one statement that needs revision; give a concrete rewrite, not only a score.

# Submission checklist

- Named stakeholder, decision, and boundary
- Three functional requirements
- Two measurable quality requirements
- One domain constraint or stated assumption
- Acceptance evidence for every requirement
- One trade-off decision record
- Peer-review comment and revision
- Requirement identifiers (FR, NFR, DR)

**Repo activity**

Use the existing fraud API fields or pasteurization stream fields as evidence for the current boundary. A requested field or endpoint that does not exist is a legitimate new requirement—label it as such.

# Wrap-up

*What should remain after the workshop?*

References and take-away message

1. Accountability | 2. Writing | 3. Specification | 4. Workshop | 5. Wrap-up

# Papers and materials

## Papers and materials


- IEEE (1998). *IEEE Std 830-1998: IEEE Recommended Practice for Software Requirements Specifications*. IEEE Computer Society. ieeexplore.ieee.org/document/720574.
- ISO/IEC/IEEE (2018). *ISO/IEC/IEEE 29148:2018: Systems and software engineering – Life cycle processes – Requirements engineering*. ISO.
- Sommerville, I. (2016). *Software Engineering*, 10th ed. Pearson. Chapter 4: Requirements engineering.
- Wiegers, K., Beatty, J. (2013). *Software Requirements*, 3rd ed. Microsoft Press.
- Vogelsang, A., Borg, M. (2019). Requirements engineering for machine learning: perspectives from data scientists. *Proceedings of the IEEE 27th International Requirements Engineering Conference Workshops (REW)*, 245–251.
- Gift, N., Deza, A. (2021). *Practical MLOps*. O'Reilly Media.
- Atlassian: user stories and acceptance criteria.

# Take-away message

A requirement is a commitment about observable system behaviour:
state the condition, the measurable target, and the evidence,
record it in a structured specification,
and trace it to a test and an operational check.

**Thank you for your attention.**

Next: Class 23 — Process models and agile foundations.